

I would like a prompt to try on Fable which fits within my plan limitations. My goal would be for an architecture for an LCC node feature that uses OpenLcbCLib as I do on my RPi Pico boards which I have been working on with Claude Code. This feature would be to imbed an interpreter to execute programs loaded into NVM through LCC datagrams, the CDI, or some other means. These programs would be in a generally understood language like MicroPython or Basic. The RRCirKits Tower Q++ uses an STL subset where the code is loaded through the CDI. The programs would be procedural and allow the user to program logic with events as the primary inputs and outputs. I would like a plan that I could use with Claude Code over time to develop this solution.

Prompt for Fable: LCC Node Embedded Scripting Interpreter — Architecture

Paste everything below the line into Fable as a single message. It's written to get one thorough architecture document back rather than a multi-turn interview, since the goal is a reference, you can carry into Claude Code afterward.

Context

I build custom LCC (OpenLCB / NMRA S-9.7) nodes on Raspberry Pi Pico boards (RP2040 and RP2350) using **OpenLcbCLib** as the protocol stack. My nodes already handle event production/consumption, CDI-based configuration, and NVM storage (onboard flash and/or external I2C EEPROM/FRAM). I do the implementation myself in Claude Code over many sessions — what I need from you right now is a design, not code.

Prior art I want you to reason against explicitly: RR-CirKits' Tower LCC+Q is a 16-line LCC I/O node with a built-in "Logic Engine" that runs a Statement List (STL) language — a subset of Siemens S7-x. The STL logic is authored entirely inside the CDI: 16 logic groups, each holding a 256-character "MultiLine" of text (4,096 characters of logic total across the node), edited live through a JMRI CDI tool and stored in the node's own NVM. It's proof that a real, deployed LCC node can host a live interpreter driven entirely by CDI-resident text with no separate firmware compile step. I want to match or exceed that capability but with events as first-class inputs/outputs to the language rather than just line state, and via MicroPython-like or BASIC-like syntax instead of an S7 dialect, since that's more approachable for the layout-building audience I'd be supporting.

Goal

Design an architecture for embedding a small interpreter into LCC node firmware (RP2040/RP2350, OpenLcbCLib) such that:

- User-authored programs live in NVM and persist across power cycles and reboots.
- Programs can be loaded/updated over the LCC network — via datagrams, CDI memory-access writes, a stream transfer, or whatever combination makes sense — without reflashing firmware.
- Programs are procedural (sequential statements, loops, conditionals, variables), not declarative ladder logic.

- LCC Events are the primary I/O surface: received events are how the outside world drives inputs into a running program; a program calling something like `produce(event)` is how it acts on the world. Local GPIO/sensor access should also be possible, but events are the design center.
- The language is something a hobbyist could plausibly read and write — a constrained subset of MicroPython or BASIC, or a small custom language if you think that's the better call given the constraints.

What I want from you

Work through the following and produce one structured architecture document. Where a question has a genuinely close tradeoff, give me your recommendation with reasoning rather than just listing options — I can push back on specific calls, but I'd rather start from a stated position than a menu.

1. **Language / VM choice.** Compare realistic candidates given RP2040 (264KB SRAM, typically 2MB flash) and RP2350 (520KB SRAM) constraints: a MicroPython subset (e.g. trimming an existing tiny Python VM vs. writing a minimal one), a tiny BASIC dialect, a Forth-style threaded-code VM, or a small custom bytecode VM/language purpose-built for event-driven procedural logic. Consider flash/RAM footprint, determinism of execution time, ease of writing a byte-code compiler that could run on-device or host-side, and how each maps naturally onto "wait for event," "produce event," "read/write variable" primitives. Name existing open-source implementations worth adapting rather than writing from scratch, if any fit.
2. **Execution model.** How the interpreter integrates with OpenLcbCLib's event callback flow — interrupt-context event arrival feeding a cooperative interpreter loop vs. a polling model, how a program blocks/resumes waiting on an event, watchdog/runaway-loop protection, and what happens to in-flight state across a node reboot.
3. **NVM layout.** How script storage coexists with existing CDI-configured node parameters and OpenLcbCLib's own NVM usage — segment layout, versioning/checksums, size budget per program, and whether multiple independent programs (e.g. per your "primary inputs and outputs" model, maybe one program per logical device or one big program) make more sense.
4. **Loading/transport mechanism.** How a program actually gets from a PC/JMRI tool onto the node: LCC's Configuration Memory Access datagram protocol (as CDI editing already uses), a custom datagram type, the Stream protocol for larger payloads, or CDI string elements a la Tower LCC+Q's MultiLines. Weigh against the Tower LCC+Q approach specifically — what it gets right for a CDI-only, tool-friendly workflow, and where a datagram/stream approach would do better for a more expressive language.
5. **Event binding model.** How the language names and binds to specific 64-bit Event IDs — variable-to-event mapping, whether binding is done in the script text itself or via separate CDI event-learn fields referenced by the script (matching how LCC nodes normally expose event configuration for learning/binding in JMRI).
6. **Phased development roadmap for Claude Code.** Break the build into milestones I could work through incrementally over many Claude Code sessions — e.g. language/opcode spec, host-side interpreter + test harness before touching hardware, OpenLcbCLib event hook integration, NVM storage,

datagram/CDI loading path, CDI XML for configuration, then hardware bring-up. Order it so each milestone is independently testable.

7. **Open risks.** Flag anything you think is a real gotcha — memory budget under pressure, execution-safety (infinite loops, watchdog resets mid-script), debugging support on a headless embedded node, and how a language-spec change would be migrated across nodes already running an older version of a script.

Format

One structured document, organized by the sections above. Skip code — this is a design document I'll turn into implementation work separately. If something is genuinely underspecified and would change your recommendation, ask a single clarifying question before writing; otherwise state your assumption and proceed.

Embedded Event-Scripting Engine for OpenLcbCLib Nodes on RP2040/RP2350

Architecture Design Document — v1

Target: RP2040 (264KB SRAM / 2MB QSPI flash) and RP2350 (520KB SRAM), bare-metal C firmware built on OpenLcbCLib. Prior art baseline: RR-CirKits Tower LCC+Q STL Logic Engine (16 × 256-char CDI MultiLines, S7-STL subset, edited live via any generic CDI tool).

Stated assumptions, in lieu of a clarifying question: OpenLcbCLib is run in its normal cooperative main-loop configuration (CAN RX may be interrupt-assisted, but protocol callbacks fire in main-loop context); nodes are CAN-attached (so Stream support across the installed tool base cannot be assumed); and the audience authoring scripts is layout builders using JMRI, not developers with a toolchain. All three assumptions shape the recommendations below, and I flag where a different assumption would change the call.

1. Language / VM Choice

The constraint that actually decides this

The Tower LCC+Q's most important property is not its language — STL is frankly user-hostile — it is that **the only tool a user ever needs is a generic CDI editor**. Text goes into a JMRI textarea, gets written to config memory over standard datagrams, and the node does the rest. No installer, no compiler download, no version skew between a host tool and firmware. Any design that requires a host-side compiler forfeits that, and for a hobbyist audience that forfeiture is expensive. So the deciding constraint is: **the compiler must run on the node**, taking source text from NVM and producing bytecode at commit/boot time. That immediately re-ranks the candidates.

Candidates

MicroPython (embed port or trimmed fork). The official RP2 port proves it runs on this silicon, and the embed port exists precisely for hosting MP inside foreign firmware. But even minimized builds want roughly 100–

300KB of flash and a GC heap of 16KB+ before your program does anything, GC pauses make execution timing non-deterministic alongside a CAN stack you must service continuously, and the compiler+runtime is a large foreign codebase to co-own. It maps poorly onto "wait for event" without building an async layer on top. On RP2040 sharing 264KB of SRAM with OpenLcbCLib buffers, CAN queues, and your existing application, this is the heavy option. Viable on RP2350 if you later want a "pro tier," but wrong as the core.

Tiny BASIC family (uBASIC, TinyBasic Plus, MY-BASIC). Adam Dunkels' uBASIC is a few KB and interprets source text directly — genuinely tiny, easy to bolt primitives onto, MIT-style licensed. Weaknesses: line-numbered GOTO dialects are dated even for hobbyists; direct source interpretation re-tokenizes on every pass (fine at this scale, but it makes per-statement execution cost irregular); and none has a native concept of suspending on an external event — you'd bolt on a coroutine mechanism anyway. MY-BASIC is more modern (structured, no line numbers) but ~10× larger and heap-centric.

Forth-style threaded VM. Technically the best fit for the silicon — tiny, deterministic, trivially extensible with EVENT/PRODUCE words, and the "compiler" is almost free. Disqualified on audience grounds: RPN and stack juggling is a harder sell to layout builders than the S7 STL you're trying to improve on. Only worth revisiting if you personally would be the only author.

Purpose-built small language + bytecode VM, compiled on-device. A single-pass recursive-descent compiler for a small structured grammar (no line numbers; if/elif/else, while, integer/boolean variables, named event handlers, wait with timeout) is on the order of 5–10KB of code and a few KB of working RAM — very much writable and maintainable in Claude Code sessions. The VM is a small switch-dispatch loop over a compact ISA (~30–40 opcodes), giving you exact control over instruction budgets, suspension points, and event primitives as first-class opcodes rather than library calls.

Recommendation

Build a purpose-built language — call it EventScript here — with a Python-flavored surface syntax, compiled on the node from CDI-resident source text into bytecode held in RAM. Reasoning: it is the only option that simultaneously (a) preserves the CDI-tool-only workflow, (b) fits comfortably in the RP2040 budget with room to spare, (c) gives deterministic, budgetable execution next to a live CAN stack, and (d) makes events first-class in the grammar instead of bolted on. The cost — owning a compiler — is real but small at this grammar size, and it's the kind of well-specified, exhaustively-testable component Claude Code is good at building incrementally.

Surface style matters for adoption: prefer Python-ish structure (indentation optional; end-delimited blocks are easier to compile and more forgiving in a textarea) with a deliberately tiny type system: integers, booleans, and named events. A flavor of what a user would write:

```
on ev_east_approach:
  if gate_state == 0 then
    produce(ev_gate_lower)
    gate_state = 1
    wait ev_island_clear timeout 30s
    produce(ev_gate_raise)
    gate_state = 0
  end
```

end

every 500ms:

if flashing then toggle(pin_led) end

end

Do not persist bytecode. Source text in NVM is the single source of truth; compile at boot and on commit. Compilation of a few KB of source is milliseconds on a 133–150MHz core, and it means the bytecode ISA can evolve freely between firmware versions (see §7).

If you want a running start rather than a clean sheet, uBASIC is worth reading as a skeleton for the tokenizer/expression evaluator shape, but I'd treat it as reference, not foundation — the coroutine suspension model in §2 wants to be designed in, not retrofitted.

2. Execution Model

Integration with OpenLcbCLib

OpenLcbCLib's application surface is a cooperative main loop: CAN frames may arrive under interrupt, but event-consumed callbacks and the periodic timer tick execute in loop context. The interpreter should live entirely in that same loop as one more cooperative task. Concretely:

Event inbox. The OpenLcbCLib "event consumed" callback does exactly one thing: push the matched event's index (not the 64-bit ID — the small table index from §5) into a fixed-size ring buffer and return. No interpretation happens in the callback. This keeps the callback $O(1)$ regardless of script complexity and makes the ISR/loop boundary clean if you ever move RX matching into interrupt context.

Scheduler slice. Each pass through the main loop, after servicing the protocol stack, the VM scheduler runs: drain the inbox, mark any handlers whose trigger event arrived as runnable, resume any suspended handler whose awaited event arrived or whose timer expired, then execute runnable handlers under an **instruction budget** — a hard cap of N bytecode ops per slice (start around 1,000; tune against your loop period). A handler that exhausts the budget is preempted at the next loop back-edge and resumes next slice. The CAN stack is therefore never starved by user code, by construction.

Blocking and resuming: coroutine handlers

Pure run-to-completion (the Tower model, effectively) can't express the procedural sequences you want ("lower gate, wait for island clear, raise gate"). So each handler is a lightweight coroutine: `wait <event> [timeout <t>]` saves the handler's program counter and its small evaluation stack into that handler's context block and yields. The scheduler resumes it when the awaited event index appears in the inbox or the timeout fires. Context blocks are statically allocated at compile time — the compiler knows the maximum stack depth of every handler, so there is no dynamic allocation anywhere in the VM. A reasonable budget: 8–16 concurrent handler contexts of ~64–128 bytes each; a few KB total.

Two triggers exist besides events: on startup: (runs once after compile/boot) and every `<t>`: periodic handlers driven off OpenLcbCLib's timer tick. That trio — startup, event, periodic — covers the Tower's use cases and considerably more.

Runaway protection and faults

Three layers. First, the per-slice instruction budget already guarantees loop liveness — an infinite while loop degrades into a handler that consumes its slice forever but never blocks the node. Second, a per-handler cumulative budget (e.g., 100k instructions without a wait or handler completion) trips a **script fault**: the handler is disabled, a status code and source line number are written to a read-only CDI status field, and optionally a diagnostic event is produced so JMRI can alarm on it. Third, the hardware watchdog stays owned by the firmware, not the VM — the VM can never be the component that decides whether the watchdog gets fed.

Reboot semantics

Keep this brutally simple in v1: on any reboot, all suspended handlers are cancelled, all variables reset to their declared initial values, and on startup: handlers run. In-flight waits do not survive. Add a small class of **retained variables** (retain gate_state = 0) persisted to NVM on change — rate-limited if the backing store is flash, unlimited if FRAM — for state that genuinely must survive power cycles, like a turnout's last commanded position. Do not attempt to persist coroutine continuations; the complexity-to-value ratio is terrible and the failure modes (resuming a wait for an event that fired during the outage) are worse than a clean restart.

One LCC-correctness note: because produced/consumed events are enumerated in CDI tables (§5), the firmware can answer Identify Producer/Consumer and emit the boot-time Producer/Consumer Identified reports with correct valid/invalid state from retained variables — something script-embedded event IDs would make nearly impossible. This is a second, independent argument for the §5 binding model.

3. NVM Layout

Where scripts live relative to existing config

Keep script storage **inside the standard configuration address space (0xFD)** that OpenLcbCLib already exposes for CDI-defined parameters, as a new segment appended after your existing node configuration. This is what makes the whole Tower-style workflow fall out for free: generic tools already know how to read and write this space with Memory Configuration datagrams, and JMRI renders it from your CDI XML with zero custom tooling. A custom address space (say 0xA0) is cleaner in the abstract but buys nothing until you have a custom host tool, which is explicitly a non-goal for v1.

Slot structure

Divide the script region into **fixed-size program slots** — recommend **8 slots of 2KB source each** (16KB total) as the default build, configurable at compile time. That is 4× the Tower's total logic capacity per slot, 32× overall, while staying trivial in a 2MB flash or a 32KB FRAM. Fixed-size slots cost some internal fragmentation but make the CDI XML static, the memory map stable across edits, and each slot independently rewritable.

Per-slot layout: a small header (enable flag, 16-char name, format-version byte, CRC32 of the source text, reserved bytes), the source text area, and a **read-only status block** the firmware maintains — compile result, error line/column, error message string, run state, fault code. Surfacing compile errors as a readable CDI string is the single biggest usability win over guessing why logic silently doesn't run.

Separately: a shared **event binding table** region (§5) and a small **retained variables** region, each with their own CRC.

Backing store and the XIP trap

This choice has a hardware landmine. On RP2040 (and RP2350), code executes XIP from the same QSPI flash you'd be erasing/writing; **any flash program/erase stalls all XIP execution** for the duration — sector erases run milliseconds — during which CAN frames can be dropped unless RX is fully interrupt-buffered from RAM-resident code. Since your boards already support external I2C EEPROM/FRAM, **prefer external FRAM (e.g., MB85RC series) as the script/retained-variable store**: byte-writable, effectively infinite endurance (ideal for retained variables written on every state change), no erase stalls, no wear leveling. If a board must use onboard flash, align each slot to a 4KB erase sector, buffer writes in RAM and commit whole slots, run the flash-write routine from SRAM, and accept a brief, bounded service pause at commit time only (commits are rare; retained-variable writes to flash should be coalesced and rate-limited, e.g., one commit per 10s max).

One program or many?

Multiple independent slots. Isolation is the point: a compile error or runtime fault in the crossing-gate script must not take down the signal ladder in the next slot. Slots also give users a natural per-function organization, partial updates rewrite only one sector, and per-slot status makes debugging tractable. Default to isolated variable namespaces per slot; add an explicit shared variable declaration later only if real use cases demand cross-slot state (event-based signaling between slots is the more LCC-idiomatic coupling anyway — one slot produces, another consumes).

4. Loading / Transport Mechanism

Recommendation: standard Memory Configuration datagrams into CDI-declared strings

The Tower approach — script text as string elements in config space, edited through a generic CDI tool's text areas and written via standard Memory Configuration Protocol datagram writes — is the right transport, and I'd adopt it outright with two upgrades. What it gets right: universal tool support today (JMRI, Model Railroad System, any future CDI tool), automatic chunking of large writes into 64-byte datagram payloads by the protocol layer, no custom protocol to specify or maintain, and the node needs nothing beyond the config-memory handling OpenLcbCLib already provides.

Where the Tower approach is weak and the fixes:

Commit semantics. The Tower recompiles implicitly; with larger multi-datagram writes you want an explicit boundary so the compiler never sees a half-written slot. Fix: per-slot CRC in the header plus an explicit commit trigger — a one-byte "action" field per slot (CDI can render it as a button-like selector: Compile & Run / Stop / Revert) written last. The Memory Config "update complete" notification, where the tool sends it, is a belt-and-suspenders recompile hint, but don't depend on tools sending it.

Editing ergonomics. A JMRI text area with no syntax highlighting and per-field size limits is clunky at 2KB.

Accept it for v1 — it is exactly the Tower's UX and it works — and treat a nicer editor as a later, optional layer: because the transport is plain memory-space writes, a small host-side helper (or a JMRI script) can round-trip

slot contents without any node-side changes. The architecture decision is that **any such tool is sugar, never a requirement.**

Rejected alternatives

Stream protocol: the payloads ($\leq 2\text{KB}$) don't need it, and Stream support across CAN-attached nodes and tools is spotty enough that requiring it would shrink your compatible-tool set to roughly zero for no benefit. **Custom datagram types:** all cost (spec, tool support, forward compatibility) and no capability the memory protocol doesn't already give you. **Firmware-upgrade-style transfer (space 0xEF + freeze):** wrong semantics — scripts are configuration, not firmware, and freeze/unfreeze reboot behavior would fight the live-edit workflow.

5. Event Binding Model

Bind events in CDI tables; reference them by name in scripts. Each slot's CDI section (or one node-level table — per-slot is tidier) carries N event entries, each an 8-byte EventID element plus a short name string, e.g. 16 entries per slot. JMRI renders EventID fields with its standard event widgets — copy/paste from other nodes, and critically the **event capture/"learn" workflow**, where the user presses the fascia button or throws the turnout and captures the resulting event straight into the field. Scripts then refer to `ev_east_approach` by the name given in the table.

Why this beats event IDs in script text: it keeps 16-hex-digit IDs (with dotted-form transcription errors) out of user code; re-binding to different hardware requires no code edit — the same crossing-gate script drops onto a second crossing by rebinding its table; the firmware knows the complete producer/consumer sets **without parsing user code**, so OpenLcbCLib event registration, Identify handling, and boot-time Producer/Consumer Identified reports (§2) work correctly and immediately, even for slots that failed to compile; and it matches the configuration idiom every LCC user already knows from every other node.

Direction (produced vs. consumed vs. both) can be inferred by the compiler from usage (`on X: / wait X` $\Rightarrow$ consumed; `produce(X)` $\Rightarrow$ produced) and written into the registration tables at commit — no need to burden the user with declaring it. Allow a literal-EventID escape hatch in the grammar for power users, but document the table as the intended path.

Linkage mechanics: at compile time, names resolve to table indices; the bytecode carries indices only. Renaming or re-binding an entry therefore requires recompiling dependent slots — handled automatically since commit recompiles, but a table edit should also mark dependent slots for recompile (simplest rule: any table write recompiles the slot).

6. Phased Development Roadmap for Claude Code

Ordered so each milestone is independently testable, and so hardware enters as late as possible — the compiler and VM are pure portable C and should be bulletproof on the host before they ever see a Pico. Milestones 1–3 are also the cheapest place to change your mind about language design; treat the spec as frozen only after M3.

M1 — Language and VM specification (paper only). Grammar (EBNF), the ~30–40 opcode ISA, handler/coroutine semantics, fault codes, slot header format, event-table format, CDI XML sketch. Deliverable: a spec document plus 5–6 worked example programs (crossing gate with island timeout, ABS signal ladder, turnout interlock, fascia-button dispatcher, startup state restoration) hand-checked against the grammar. This document becomes the standing context file for every subsequent Claude Code session.

M2 — Host-side compiler + VM, CLI harness. Portable C99, zero hardware dependencies, malloc-free core (static pools sized by compile-time constants). A CLI that compiles a source file, dumps bytecode listings, and runs it against a scripted event/timer stimulus file, logging produced events. Unit tests for the tokenizer, parser (including every error path — error line reporting is a feature, test it), and each opcode. Golden-file tests for the example programs from M1.

M3 — Runtime semantics hardening on host. Coroutine suspend/resume across wait, timeout arbitration, instruction budgets and fault trips, inbox overflow policy, retained-variable change tracking. Add a fuzzer over the compiler (random token streams must never crash it, only produce errors) — an afternoon of Claude Code work that pays for itself the first time a user typos a script.

M4 — Pico bring-up without LCC. Link the engine into a minimal RP2040/RP2350 firmware: scripts baked in as string constants, GPIO/timer primitives wired, instruction budgets tuned against a real main-loop period, SRAM/flash footprint measured and recorded. Validates the no-allocation claim and the slice scheduler on real silicon.

M5 — OpenLcbCLib integration. Event-consumed callback → inbox; produce → the library's produce path; event-table-driven registration; Identify Producer/Consumer and boot-time Identified reports from retained state. Test node on the bench against JMRI: send events from the JMRI event tool, watch script-produced events appear.

M6 — NVM slots + CDI. Slot layout on FRAM (and the flash fallback with RAM-resident write path), CRC verification, compile-on-commit, status/error fields, full CDI XML. Test: edit a script in a JMRI textarea, commit, watch the status field report success or a precise error line; power-cycle and confirm persistence and retained variables.

M7 — Workflow polish and user documentation. Multi-slot fault isolation testing, event-table learn-mode round trips, the Revert action, a user-facing language reference with the example-program library, and a "porting a Tower LCC+Q STL group to EventScript" worked example — a good adoption on-ramp for exactly your audience.

M8 — Layout soak. Run real jobs (your paper-mill traffic logic is a natural candidate) on live hardware for weeks; capture faults via the diagnostic event; iterate budgets and inbox depths from field data.

7. Open Risks

The XIP/flash-write stall is the sharpest hardware gotcha. If any target board stores scripts or retained variables in onboard QSPI flash, every erase stalls code execution unless the write path runs from SRAM and CAN RX buffering is interrupt-driven and RAM-resident. Retained variables written on every state change

would also chew flash endurance. Mitigations are in §3; the strategic mitigation is standardizing on FRAM for this feature's storage and treating onboard flash as the degraded path.

RAM budget creep on RP2040. The engine itself is small, but it sits next to OpenLcbCLib buffers, CAN queues, your existing application, and now per-handler contexts, inbox, compiled bytecode, and compiler working memory. Everything is statically sized, so make the sizing knobs explicit compile-time constants from M2 onward and track a memory map per milestone. If a squeeze comes, compiler working RAM is reclaimable (compile one slot at a time), and bytecode for disabled slots need not be resident.

Headless debugging. A wrong-but-compiling script on a node with no console is the top support burden to expect. Budget real design effort for: per-slot status strings (current handler, last event consumed, fault + line number) in read-only CDI; a `log()` statement that produces a designated diagnostic event or appends to a RAM ring buffer exposed as a readable memory region; and possibly a single-step/trace mode toggled from CDI. These are cheap on top of the architecture and transform the user experience.

Language evolution vs. deployed scripts. Because bytecode is never persisted, the ISA is free to change with firmware. Source compatibility is the real contract: grammar changes must be additive, the slot-header format-version byte gates any future breaking change (old-version slots refuse to auto-compile and report "update script syntax" rather than misbehaving), and the spec doc from M1 should carry a changelog from day one. Resist per-node dialect drift across your RP2040/RP2350 fleet — one grammar, one version number, everywhere.

Event storms and inbox overflow. A busy layout can burst events faster than one scheduler slice drains them. Define the overflow policy now (drop-oldest with a fault counter is defensible; drop-newest silently is not), size the inbox generously (it's bytes per entry), and count drops in the status block so the failure is visible, not mysterious.

Timing semantics honesty. There is no RTC and no global clock; all times are node-local relative durations, and every 500ms: has jitter bounded by the main-loop period plus budget preemption. Document this so nobody builds a fast-clock or cross-node-synchronized animation on promises the platform can't keep. If wall-clock behavior is ever wanted, consume the LCC clock protocol's events like any other producer — which the architecture already supports for free.

Scope creep toward MicroPython. Users will eventually ask for strings, arrays, and floats. The architecture tolerates growth (ISA versioning, on-device compiler), but each addition erodes the determinism and footprint story that justified the custom VM. Pre-commit to a line: integers, booleans, events, timers in v1; revisit only with field evidence. If demand for a rich language materializes, that is the moment to evaluate a MicroPython tier on RP2350-only hardware rather than growing EventScript into a worse MicroPython.

M1- draft

There's the M1 draft — full EBNF, execution semantics, a 30-opcode stack ISA with room reserved for v2, a worked crossing-gate example with its compiled listing, and the limits and error-category tables that M2's test suite hangs off of.

The decisions I made on your behalf that most deserve a second look are marked 🚧 in the doc, but here's the short version:

Explicit var declaration required. BASIC-style implicit declaration is friendlier for five minutes, then `gatestate` vs `gate_state` becomes an unfindable bug on a headless node. I chose the compile error.

One-deep pending latch on retrigger. If an event fires while its handler is mid-wait, one re-run is latched and further triggers are dropped-and-counted. Unbounded queueing and restart-on-retrigger both lose worse in the common gate/turnout cases, but this is a semantics call you should sanity-check against how you'd actually script your layout.

No wait `ev_a` or `ev_b` in v1. This is the cut most likely to hurt in practice — it's flagged as the first v1.1 candidate, and opcode space is reserved so adding it doesn't disturb the ISA.

No fused opcodes, no peephole cleverness. `x = x + 1` is four instructions and that's fine at layout-logic rates. The doc pins golden tests to execution traces rather than exact bytes specifically so the compiler stays free to improve later without invalidating the test suite.

One process suggestion for when this goes into Claude Code: put both documents in the repo (something like `docs/architecture.md` and `docs/spec.md`) and reference them from `CLAUDE.md` so every session starts grounded in the same contract — that's what makes an M1 spec pay off across a many-session build. The natural first session is the M2 tokenizer + parser with the §9 error categories as the initial test matrix.

EventScript Language Specification — v0.3 (M1 Draft)

Companion to the architecture document. This is the standing-context spec for Claude Code sessions covering M2 (host compiler + VM) onward. Everything here is a proposal to be argued with; decision points that most deserve pushback are marked 🚧.

Changelog — v0.3: fast clock reworked around a **local consumer clock** as the source of truth (generators broadcast only on start/set plus infrequent heartbeats; nodes keep their own time). `at` triggers fire from local minute crossings, not network events; registers read the local clock continuously; discontinuity policy defined in §5.6. v0.2: added fast-clock registers and the `at HH:MM:` trigger. v0.1: initial draft.

1. Design Ground Rules

One value type: **signed 32-bit integer**, two's-complement, wrapping arithmetic (deterministic, no traps on overflow; division by zero is a runtime fault). Booleans are integers — comparisons yield 0 or 1, conditionals treat any nonzero value as true. No strings, floats, or arrays in v1. No dynamic allocation anywhere: every limit in §8 is a compile-time constant of the firmware, and the compiler rejects programs that exceed them with a specific error and line number.


```

;

assignment = IDENT "=" expr ;

if_stmt    = "if" expr "then" block
            { "elif" expr "then" block }
            [ "else" block ]
            "end" ;

while_stmt = "while" expr "do" block "end" ;

event_ref  = IDENT                                # name from the slot's CDI binding table
            | "event" "(" EVENTID ")" ;          # literal escape hatch (discouraged)

expr       = or_expr ;
or_expr    = and_expr { "or" and_expr } ;        # short-circuit
and_expr   = not_expr { "and" not_expr } ;        # short-circuit
not_expr   = [ "not" ] cmp_expr ;
cmp_expr   = add_expr [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) add_expr ] ;
add_expr   = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr   = unary { ( "*" | "/" | "%" ) unary } ;
unary      = [ "-" ] primary ;
primary    = INT | DURATION | "true" | "false"
            | IDENT                                # variable read
            | "timedout" | "millis"
            | "fasthour" | "fastminute" | "fastrunning"
            | "getline" "(" expr ")"
            | "(" expr ")" ;

```

Notes: comparison is non-associative ($a < b < c$ is a compile error — almost always a bug at this audience level). `and/or` compile to branches, not opcodes, so evaluation is short-circuit by construction.

4. Name Resolution

Variables are slot-scoped statics, declared with `var/retain` before first use; assignment to an undeclared name is a compile error (catches typos, the number-one bug class in a textarea). 🐞 Alternative: BASIC-style implicit declaration on first assignment — friendlier to beginners, but silently turns `gatestate = 1` vs `gate_state = 1` into a mystery. Recommendation stands on explicit declaration.

Events resolve against the slot's CDI event-binding table by name (case-sensitive match against the table's name strings). The compiler infers direction from usage — `on X: /wait X` marks entry `X` consumed, `produce(X)` marks it produced, both is legal — and emits the consumer/producer registration sets as a compile artifact so the firmware registers with OpenLcbCLib without parsing source. An `event(...)` literal creates an anonymous internal binding; it participates in registration but not in the CDI table, which is why it's the discouraged path.

There are no handler-local variables and no user-defined functions in v1. 🐞 Both are the most likely v2 additions; the ISA below leaves room (a frame-relative load/store pair and `CALL/RET`) without renumbering.

5. Execution Semantics

5.1 Handlers and triggers

A slot's program is a set of handlers. Exactly four trigger kinds:

`on startup`: runs once after a successful compile — at boot, and again after each commit. `on <event>`: runs when the bound event is consumed. `every <period>`: runs on a node-local periodic timer, first firing one period after startup completes. `at HH:MM`: runs when the node's local fast clock reaches that time (§5.6); it enters the scheduler like any other trigger, so §5.2–5.3 apply to it unchanged.

At most **one handler per trigger** per slot: two `on ev_x`: blocks, or two `at 6:00`: blocks, are a compile error (compose within one handler instead). One event may, however, trigger handlers in several different slots — slots are independent consumers.

5.2 Run states and the pending latch

A handler is IDLE, RUNNING, or SUSPENDED (inside `wait/sleep`). If its trigger fires while it is RUNNING or SUSPENDED, one re-run is **latched**: when the current run completes, it runs once more. Further triggers while the latch is set are dropped and counted in the slot's status block. 🐞 Alternatives considered: unbounded queueing (unbounded memory, rejected), restart-on-retrigger (kills in-progress sequences — wrong default for gate logic, though a per-handler `restartable` modifier is a plausible v2), plain drop (loses the common "button pressed again during cycle" case). The one-deep latch is the smallest thing that handles real layouts predictably.

5.3 wait, sleep, timeout

`wait ev timeout t` suspends the handler until event `ev` is next consumed or `t` ms elapse, whichever is first, then sets the `timedout` register (1 on timeout, 0 on event) and resumes. `wait ev` with no timeout waits indefinitely. Only events consumed **after** suspension begins can satisfy a wait — there is no event memory. `timeout 0` is a compile error (use `getline/variables` to poll state instead). `sleep t` is pure delay; it leaves `timedout` untouched. `timedout` is per-handler and persists until the handler's next `wait`.

Waiting on one of several events is deliberately absent from v1. 🐞 It is the most tempting scope add (`wait ev_a or ev_b`); it roughly doubles suspension-record complexity. The workaround — a variable set by tiny `on` handlers, polled after a short sleep loop — is clumsy but expressible. Flagged as the first candidate for v1.1 based on how often the workaround shows up in real scripts.

5.4 Built-in registers and I/O

`millis` reads the node uptime counter (wraps at $2^{32}\text{ms} \approx 49.7$ days; subtraction of two readings is wrap-safe by two's-complement arithmetic — worth one line in the user docs). `getline(n) / setline(n, v)` read and write logical I/O lines 1..N; the firmware owns the mapping from line numbers to physical GPIOs per board. An out-of-range line index is a runtime fault, not silent truncation.

5.5 Faults

A runtime fault disables the offending handler (not the slot, not the node), records fault code + source line in the slot status block, and optionally produces the node's diagnostic event. Fault codes: `DIV_ZERO`,

BUDGET_EXCEEDED (per-run cumulative instruction ceiling, §8), BAD_LINE_INDEX, STACK_OVERFLOW (defense in depth — the compiler's static stack sizing should make this unreachable), LATCH_DROPS (informational counter, non-fatal). Re-enabling is by recommit or reboot.

5.6 Fast clock (LCC Broadcast Time Protocol)

The node tracks one clock, selected by a node-level CDI setting (clock ID prefix; default: the well-known Default Fast Clock). **The source of truth is a local consumer clock maintained by the firmware's clock module** — synchronized whenever a Report Time / Date / Rate / Start / Stop event arrives, and advanced between broadcasts by the node's own timebase scaled by the last reported rate. This matches deployed generator behavior: real generators broadcast time on start and set, then only an infrequent heartbeat, and expect consumers to keep their own time. Nothing in the language depends on the generator emitting per-minute events. On the network side the module sends the protocol's Consumer Range Identified over the clock's time-event range and issues a sync query at boot; received events only ever *correct* the local clock.

The language surface has two facilities:

Registers (for conditions). `fasthour` (0–23), `fastminute` (0–59), and `fastrunning` (0/1), read from the local clock — continuously valid while the clock runs. Before the first synchronization after boot, `fasthour` and `fastminute` read **−1**; scripts that must be robust at power-up guard with `fasthour >= 0`. While the clock is stopped the registers hold the last time. Range conditions are the intended use, not equality tests:

```
# lamps on from 19:00 to 06:00 — range spanning midnight
if fasthour >= 19 or fasthour < 6 then ... end
# finer granularity: minutes since midnight
if fasthour * 60 + fastminute >= 6 * 60 + 30 then ... end
```

at HH:MM: trigger (for scheduling). Fired by the local clock crossing that minute boundary — a firmware-internal dispatch, never a network event consumption, so it is immune to broadcast cadence and generator implementation quality. During continuous progression the module fires every `at` handler whose minute the local clock passes, in time order, in either direction for backward-running clocks; at high rates several minutes may be crossed between scheduler slices and the resulting firings are bounded by the one-deep latch (§5.2). Triggers recur every fast day and do not fire while the clock is stopped. Time bindings are internal: they consume handler-table entries but **not** the slot's 16-entry CDI event table.

Discontinuity policy (owned by this firmware, not the generator). A received time report that disagrees with local progression is classified by the gap: a forward correction of at most **2 fast minutes** is treated as drift synchronization, and any `at` triggers in the corrected-over span fire in order — routine sync must never silently eat a scheduled action. Anything larger, in either direction, is a **discontinuity**: the clock jumps, the registers update, and no intervening `at` triggers fire. Rationale is the classic church-bell problem — replaying a burst of scheduled instantaneous actions for time that "didn't happen" is worse than skipping them. 🛑 The 2-minute threshold is a judgment call; it only needs to exceed worst-case local drift between heartbeats, and it should be a firmware constant, not user-visible.

The scripting guidance that follows, for user docs: use `at` alone for instantaneous actions (sound a horn, dispatch a train); derive *state* — lights, signals, anything "on between times" — from register range-checks or from paired `at` triggers writing a retained variable, so that a time-set converges to correct state even though the skipped triggers never fired.

The clock module is plain C over injected time reports and an injected timebase, so it is fully host-testable in M2–M3 alongside the VM: sync, drift correction, discontinuity classification, stop/start, backward rates, and midnight rollover all get golden tests before hardware.

6. Bytecode ISA

Stack machine, one operand stack per handler context, all values int32. Variable-length encoding: 1 opcode byte + 0/1/2/4 operand bytes. Jump offsets are signed 16-bit, relative to the byte after the operand. The VM decrements the slice budget on **every instruction**, so no explicit yield opcode exists and backward jumps need no special casing. 30 opcodes assigned; 0x1F–0x3F reserved for v2 (CALL/RET, frame-relative load/store, wait-any).

Op	Mnemonic	Operand	Stack effect	Notes
0x00	NOP	—	—	
0x01	PUSH8	imm8 (signed)	— → v	
0x02	PUSH16	imm16 (signed)	— → v	
0x03	PUSH32	imm32	— → v	
0x04	LOADV	var idx u8	— → v	slot variable read
0x05	STOREV	var idx u8	v → —	retained vars: same opcode; persistence is a flag on the variable table entry
0x06	LOADR	reg u8	— → v	0=timeout, 1=millis, 2=fasthour, 3=fastminute, 4=fastrunning
0x07	ADD	—	a b → a+b	wrapping
0x08	SUB	—	a b → a-b	wrapping
0x09	MUL	—	a b → a·b	wrapping
0x0A	DIV	—	a b → a/b	trunc toward 0; b=0 ⇒ DIV_ZERO fault
0x0B	MOD	—	a b → a%b	sign follows a; b=0 ⇒ fault
0x0C	NEG	—	a → -a	
0x0D	EQ	—	a b → (a==b)	result 0/1

Op	Mnemonic	Operand	Stack effect	Notes
0x0E	NE	—	$a\ b \rightarrow (a \neq b)$	
0x0F	LT	—	$a\ b \rightarrow (a < b)$	
0x10	LE	—	$a\ b \rightarrow (a \leq b)$	
0x11	GT	—	$a\ b \rightarrow (a > b)$	
0x12	GE	—	$a\ b \rightarrow (a \geq b)$	
0x13	NOT	—	$a \rightarrow (a == 0)$ logical not	
0x14	JMP	rel16	—	
0x15	JZ	rel16	$a \rightarrow \text{—}$	jump if $a == 0$
0x16	JNZ	rel16	$a \rightarrow \text{—}$	jump if $a \neq 0$
0x17	PRODUCE	evt idx u8	—	enqueue Producer event; index into slot binding set
0x18	WAITE	evt idx u8	$t \rightarrow \text{—}$	suspend; t = timeout ms, 0 = infinite; sets timedout on resume
0x19	SLEEP	—	$t \rightarrow \text{—}$	suspend t ms; timedout untouched
0x1A	SETLINE	—	$n\ v \rightarrow \text{—}$	write logical line; bad $n \Rightarrow$ fault
0x1B	GETLINE	—	$n \rightarrow v$	read logical line; bad $n \Rightarrow$ fault
0x1C	LOG	line# imm16	$v \rightarrow \text{—}$	append (source line, v , millis) to diagnostic ring
0x1D	STOP	—	—	end handler run normally (checks pending latch)
0x1E	HALT	—	—	compiler-emitted end-of-handler; same as STOP

Lowering conventions: `and/or` compile to JZ/JNZ short-circuit chains (no AND/OR opcodes). `if/elif/else` and `while` are conventional structured lowering. `wait ev` without timeout compiles to `PUSH8 0; WAITE ev`. Duration literals are ordinary integer pushes. `x = x + 1` is `LOADV x; PUSH8 1; ADD; STOREV x` — no fused increment in `v1`; measure before optimizing.

6.1 Compiled slot image (RAM-resident, never persisted)

```
SlotImage
  magic      u16      0xE5C0
  isa_version u8       1
```

```

flags          u8
var_count      u8          ≤ 32; parallel var table: {init value i32, RETAIN flag}
handler_count  u8          ≤ 16
code_size      u16
handlers[]     each: { trigger u8 (0=STARTUP,1=EVENT,2=PERIODIC,3=FASTTIME),
                      param u16 (event idx | period ms low word - period
                                stored as u32 in a side table if > 65535 |
                                FASTTIME: hour×256+minute, matching the protocol's
                                time-event encoding),
                      entry_pc u16,
                      max_stack u8 }      # computed statically by the compiler

code[]         bytecode

```

`max_stack` per handler is the compiler's proof obligation: it must compute the exact worst-case operand stack depth (straight-line analysis suffices — the grammar has no recursion and expressions have static depth) and reject any handler exceeding the §8 cap. The VM's `STACK_OVERFLOW` check is belt-and-suspenders, not the enforcement.

7. Worked Example with Listing

Source (crossing gate, using binding-table names):

```

retain gate_down = false

on ev_approach:
  if not gate_down then
    produce(ev_gate_lower)
    gate_down = true
    wait ev_island_clear timeout 30s
    if timedout then log(1) end
    produce(ev_gate_raise)
    gate_down = false
  end
end

```

Compiled listing for the handler (annotated; offsets illustrative):

```

0000  LOADV    0          # gate_down
0002  JNZ      +0x1C       # if not gate_down ... (NOT folded into inverted branch)
0005  PRODUCE   1          # ev_gate_lower
0007  PUSH8    1
0009  STOREV   0          # gate_down = true
000B  PUSH32   30000
0010  WAITE    2          # ev_island_clear, 30s
0012  LOADR    0          # timedout
0014  JZ       +0x05
0017  PUSH8    1
0019  LOG      line=9
001C  PRODUCE   3          # ev_gate_raise
001E  PUSH8    0
0020  STOREV   0          # gate_down = false
0022  HALT

```

(The compiler is free to fold `not` into branch polarity as shown; golden tests in M2 should assert semantics via execution traces, not exact byte sequences, so peephole choices stay unconstrained.)

8. Limits (firmware compile-time constants, v1 defaults)

Limit	Default	Rationale
Program slots	8	§3 of architecture doc
Source per slot	2,048 bytes	one FRAM/flash-sector-aligned slot
Bytecode per slot	4,096 bytes	RAM budget; ~2:1 worst-case expansion observed target
Variables per slot	32	u8 index headroom to 255
Retained vars per slot	8	NVM write-coalescing budget
Event bindings per slot	16	matches CDI table
Handlers per slot	16	
Operand stack per handler	16 entries (64 B)	compiler-enforced
Handler contexts (node)	32	8 slots × avg 4 concurrent
Slice budget	1,000 instructions	tune in M4
Per-run budget	100,000 instructions	BUDGET_EXCEEDED fault
Event inbox depth	32 entries	drop-oldest + counter on overflow

9. Compile Errors (categories for M2 test coverage)

Every error carries line and column. Categories: lexical (bad character, identifier too long, integer out of range, malformed EVENTID); syntax (per-production expected-token messages — invest here, this is the user's primary feedback channel); name (undeclared variable, unknown event name, duplicate declaration, duplicate handler for trigger, assignment to register/keyword); semantic (non-constant initializer or period, `timeout 0`, chained comparison, `at` hour/minute out of range, duplicate `at` time); resource (any §8 limit, with the limit named in the message: "too many variables (max 32)").

Status-block contract: on failure the slot reports `COMPILE_ERR`, line, column, and a ≤64-char message; on success, `RUNNING`, bytecode size, and handler count. This is the entire debugging surface a headless user gets — treat error message quality as a feature with tests, not a byproduct.

10. Deferred to v2 (recorded so scope stays pinned)

Wait-any (`wait ev_a or ev_b`) — first in line, pending field evidence (§5.3). User-defined functions and handler-local variables (ISA space reserved). `restartable` handler modifier. Cross-slot `shared` variables. Fast-clock extensions: sub-minute language granularity (`at HH:MM:SS`, the local clock already tracks finer than it exposes), a `fastrate` register, date/day-of-week registers, and per-script clock selection for multi-clock layouts (§5.6). Fused increment/decrement and other peephole opcodes — only after profiling shows dispatch overhead matters, which at layout-logic rates it almost certainly does not.